

UNIVERSIDAD TECNOLÓGICA DEL PERÚ
FACULTAD DE INGENIERÍA
CARRERA DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

AVANCE DE PROYECTO FINAL 2

SISTEMA WEB DE INVENTARIO PARA D'PAO COOKIES

DESARROLLO WEB INTEGRADO
SECCIÓN 20061 · CICLO 2026-II

DOCENTE
Mtro. César Augusto Cárdenas Latorre

INTEGRANTE (GRUPO N.° 13)
• ANDERSON ENRIQUE JUNIOR CASAS GONZALES | U22244177

2026

Repositorio GitHub: https://github.com/Enrqgonzales/dpao-cookies-web/tree/apf2-frontend

Índice

Introducción y alcance de la Unidad 2....3
Objetivo del avance....3
Requerimientos de la Unidad 2 y resultado....3
Punto de partida y límites....4
Contexto real de D'PAO Cookies....5
Registro actual y descuadre de caja....7
Arquitectura de software y patrones de diseño....8
Patrón MVC + DAO con BaseServlet y servlets modulares....9
Capa DTO: justificación y estructura....11
Patrón Façade: InventarioFacade....12
Diagrama conceptual de capas....14
Módulo transaccional de Kardex....16
Atomicidad transaccional JDBC....16
Bloqueo pesimista y trazabilidad de saldos....18
Saldo no negativo y ajustes físicos....20
Capa de presentación con JSTL 1.2 y EL....22
Erradicación total de scriptlets....22
JSTL, EL y formateo regional....23
Verificación, aseguramiento de calidad y pruebas....24
Resumen de las 41 pruebas automatizadas....25
Concurrencia multihilo y rollback forzado....27
Pruebas HTTP de humo....28
Evidencias del despliegue y ejecución....29
Pruebas y control de versiones....32
Implementación con JSF 2.3 y Bootstrap 5....33
Configuración de FacesServlet, web.xml y faces-config.xml....34
Diseño de interfaces con Bootstrap 5....36
Vistas Facelets, Managed Beans y Expression Language....36
Navegación JSF y tabla de datos dinámica....40
Pruebas y repositorio de la interfaz JSF....41
Conclusiones técnicas y ruta al Proyecto Final....42
Matriz de cumplimiento de la rúbrica....43
Pendientes de evolución....45
Referencias....46

Sistema web de inventario para D'PAO Cookies: Avance de Proyecto Final 2

Introducción y alcance de la Unidad 2

D'PAO Cookies elabora y comercializa galletas artesanales. En el APF1 se entregó un prototipo web Java (Servlets, JSP y JDBC) que centraliza insumos y productos y registra las variaciones de stock en MySQL. La Unidad 2 exige que esa base evolucione hacia una aplicación transaccional organizada con patrones de diseño empresariales, con reglas de negocio explícitas y pruebas automatizadas que demuestren la atomicidad de las operaciones. El código fuente se encuentra en la rama apf2 del repositorio del proyecto (Casas Gonzales, 2026).

Objetivo del avance

Reorganizar la aplicación en capas desacopladas —controladores, fachada de negocio, objetos de transferencia y acceso a datos—, garantizar que todo movimiento de Kardex sea atómico y trazable, eliminar el código Java de las vistas y verificar el resultado con pruebas unitarias, de integración y HTTP.

Requerimientos de la Unidad 2 y resultado

La Tabla 1 relaciona cada requerimiento técnico del enunciado con el resultado comprobado en esta entrega.

Tabla 1

Requerimientos técnicos del APF2 y estado comprobado

Requerimiento	Resultado comprobado
MVC + DAO	Cuatro servlets modulares sobre BaseServlet; ningún servlet importa un DAO. Los DAO devuelven entidades tipadas en lugar de mapas genéricos.
DTO	Seis clases en pe.com.dpaocookies.dto; las vistas reciben exclusivamente DTO.
Façade	InventarioFacade es el único punto de entrada al negocio: valida, orquesta, mapea y traduce excepciones.
Kardex transaccional	setAutoCommit(false), SELECT … FOR UPDATE, commit() y rollback(), saldos anterior y posterior, rechazo de saldos negativos y ajustes por conteo.
JSTL y EL	Cero scriptlets; <scripting-invalid>true</scripting-invalid> lo impone en el contenedor.
JSF y Bootstrap	Rama apf2-frontend: FacesServlet, faces-config.xml, cinco vistas Facelets con Bootstrap 5 local, siete Managed Beans y h:dataTable con datos de MySQL; 92 pruebas aprobadas.
Calidad	41 pruebas JUnit aprobadas (0 omitidas) contra MySQL real, BUILD SUCCESS y 55 solicitudes HTTP verificadas.

Punto de partida y límites

Se conservan el esquema relacional sql/schema.sql del APF1, la autenticación con cuenta externa, la protección CSRF y el patrón Post/Redirect/Get. Siguen fuera del alcance las cuentas múltiples y roles, el responsable individual por movimiento, las recetas con consumo automático, los lotes y la caducidad, la paginación, los comprobantes de venta y el despliegue público con HTTPS.

Contexto real de D’PAO Cookies

Para este avance se levantó información directamente con el dueño de D’PAO Cookies. El negocio elabora galletas artesanales de gran tamaño que vende en tienda, en una vitrina organizada por sabor, y por delivery mediante packs; además, promociona sus productos con campañas en redes sociales. La Figura 1 muestra la identidad de marca y el punto de venta, y la Figura 2, el producto y su empaque.

Figura 1

Identidad de marca y vitrina del punto de venta



Nota. Logotipo y fotograma del video de la tienda proporcionados por el dueño el 9 de octubre de 2026.

Figura 2

Galletas y caja de empaque de la marca



Nota. Fotografías proporcionadas por el dueño de D'PAO Cookies (comunicación personal, 9 de octubre de 2026).

El catálogo vigente tiene 16 sabores agrupados en tres categorías, que también fijan el precio de los packs de delivery (Tabla 2). Los insumos que el dueño considera críticos son harina, azúcar rubia, azúcar blanca, mantequilla, chispas de chocolate, cobertura de chocolate, manjar, maicena y queso crema.

Tabla 2

Catálogo real de galletas y precios

Categoría	Sabores y precio unitario (S/)	Pack de 2, 4 y 6 (S/)
Clásico	Chispas, Chunky y Chin Chin: 7,00; Chocolate: 8,00	12 · 22 · 30
Edición	Red Velvet, Monster, Oreo, Marshmallow y Bom o Bom: 8,50; Cinnamon Roll, Arándano, Limón y Maracuyá: 9,00	16 · 30 · 42
Premium	Nutella y Snicker: 11,00; Pistacho: 15,00	20 · 40 · 58

Nota. Precios unitarios informados por el dueño. Los packs se venden solo por delivery y combinan sabores de la misma categoría.

Registro actual y descuadre de caja

Hoy cada punto de venta usa una hoja de Excel de control diario (Figura 3). La vendedora registra el stock inicial, la reposición y el stock final de cada sabor, y marca cada venta con su medio de pago: E (efectivo), Y (Yape), P (Plin) o T (transferencia). Al cierre declara el efectivo y el Yape recibidos. Según el dueño, su mayor problema es lograr que el stock y el dinero de cada tienda cuadren con las galletas vendidas.

Figura 3

Hoja de control diario del punto de venta Plaza Mayor (agosto)

D'PAO Cookies - Control Diario

AGOSTO — Punto de venta: PLAZA MAYOR — Trabajador:

Inventario del Día

	Stock inicial	Reposición	Total disponible	stock final	Ventas en efectivo (E) o yape (Y), plin(P), transferencia (T) 1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	Vendidas	PRECIOS DE AGOSTO	Venta de cada sabor
	0	0	0																				
Chispas	5	5	10	6	E	Y	Y	Y													4	7	28
Chin Chin		6	6	3	Y	Y	Y														3	7	21
Chunky	4	6	10	1	Y	Y	Y	Y	Y	Y	Y	Y	Y								9	7	63
Chocolate	1	4	5	4	E																1	8	8
Monster	2	4	6	4	Y	E															2	8.5	17
Red Velvet	1	7	8	5	E	Y	Y														3	8.5	25.5
Oreo	3	4	7	5	Y	Y															2	8.5	17
Marshmallow	3	0	3	3																	0	8.5	0
Bom o Bom	5		5	3	Y	E															2	8.5	17
Limón	4		4	0	Y	Y	Y	Y													4	9	36
Maracuyá	2	1	3	1	E	Y															2	9	18
Arándano	3		3	1	R	Y															2	9	18
Cinnamon Roll	3		3	0	Y	Y	Y														3	9	27
Nutella	3	5	8	5	Y	Y	Y														3	11	33
Snicker			0																		0	11	0
Pistacho	1	4	5	2	Y	Y	Y														3	15	45
BEBIDAS, POSTRES, OTROS																							
Postres			0																		0	9	0
Chicolak			0																		0	3	0
Shake			0																		0	6	0
GALLETAS SOBRANTES				0	TOTAL DE GALLETAS VENDIDAS																0	TOTAL	364.5

Control de dinero	
Método de pago	Monto
Efectivo	41.5
Yape	315
Plin	
Total recibido	356.5

OBSERVACIONES

19 de caja(mamá) 10 delivery

ñ

Nota. Captura proporcionada por el dueño (comunicación personal, 9 de octubre de 2026); se difuminó el nombre de la trabajadora.

Al recalcular la jornada registrada se obtiene un descuadre de S/ 17,00 (Tabla 3). El Yape coincide, pero faltan S/ 8,00 en efectivo y una venta de Arándano marcada con un código inexistente (R) no se suma en ningún medio de pago. La hoja no lo advierte: su total solo suma las ventas marcadas como efectivo o Yape, y las celdas de galletas vendidas y sobrantes muestran cero.

Tabla 3

Cuadre de la jornada registrada en la hoja de control

Concepto	**Monto (S/)**
Ventas según inventario (43 galletas por su precio)	373,50
Ventas marcadas como Yape	315,00
Yape declarado	315,00
Ventas marcadas como efectivo	49,50
Efectivo declarado	41,50
Venta con código de pago inválido (R)	9,00
Diferencia entre lo vendido y lo recibido	**17,00**

Nota. Elaboración propia a partir de la Figura 3.

El Kardex transaccional de este avance resuelve la parte del inventario: cada venta descuenta stock de forma atómica, con saldo anterior y posterior, y no admite saldos negativos. La parte del dinero, que exige asociar cada venta a un punto de venta, una vendedora y un medio de pago de una lista cerrada, y conciliar lo esperado con lo declarado al cierre, se prioriza para la siguiente iteración (véase Pendientes de evolución).

Arquitectura de software y patrones de diseño

El código se organiza en paquetes por responsabilidad bajo pe.com.dpaocookies, siguiendo el catálogo de patrones para aplicaciones Java empresariales (Alur et al., 2003). Cada capa depende solo de la inmediatamente inferior, y los objetos que cruzan las fronteras son DTO (hacia la vista) o entidades (hacia la persistencia), como resume la Tabla 4.

Tabla 4

Paquetes y responsabilidades

Paquete	Clases	Responsabilidad
controller	BaseServlet, TableroServlet, InsumoServlet, ProductoServlet, KardexServlet, AccesoServlet, SeguridadFilter	Recibir HTTP, convertir parámetros, delegar en la fachada y elegir la vista.
facade	InventarioFacade, MapeadorDTO	Reglas de negocio, orquestación de DAO, mapeo entidad → DTO y traducción de errores.
dto	InsumoDTO, ProductoDTO, CategoriaDTO, MovimientoDTO, SolicitudMovimientoDTO, ResumenTableroDTO	Transferir datos entre capas sin exponer entidades.
dao	InsumoDAO, ProductoDAO, CategoriaDAO, KardexDAO	SQL parametrizado y transacciones JDBC.
model	Insumo, Producto, Categoria, MovimientoKardex, TipoMovimiento, ClaseItem	Entidades de tabla y enumeraciones de dominio.
exception	AplicacionException, NegocioException, StockInsuficienteException, ServicioException	Errores con mensaje apto para el usuario.
util	ConexionBD, ProveedorConexion, Validacion	Conexión externalizada y reglas de formato compartidas.

Patrón MVC + DAO con BaseServlet y servlets modulares

El PanelServlet del APF1 atendía tres rutas con lógica mezclada. Se reemplazó por un servlet por módulo, todos derivados de BaseServlet, que concentra las tareas comunes: conversión de parámetros a BigDecimal, int o LocalDate con mensajes de error

comprensibles, mensajes de un solo uso y redirección GET tras cada POST. El patrón Data Access Object aísla el SQL en clases dedicadas (Alur et al., 2003), y una búsqueda en el paquete controller confirma que ninguna clase importa pe.com.dpaocookies.dao. La Tabla 3 muestra qué operación de la fachada invoca cada ruta y la Figura 4, el controlador del Kardex.

Tabla 5

Rutas, controladores y operaciones de la fachada

Ruta	**Servlet**	**GET → fachada**	**POST → fachada**
/tablero	TableroServlet	obtenerResumen()	—
/insumos	InsumoServlet	listarInsumos(), listarCategorias()	registrarInsumo(), ajustarStockInsumo()
/productos	ProductoServlet	listarProductos(), listarCategorias()	guardarProducto()
/kardex	KardexServlet	listarInsumos(), listarProductos(), consultarKardex()	registrarMovimiento()
/acceso, /salir	AccesoServlet	Formulario de acceso	Credenciales externas; invalidación de sesión

Figura 4

Método doPost de KardexServlet: el controlador arma el DTO y delega en la fachada

```
try {
    String[] item = texto(request, "item").split(":", 2);   // "PRODUCTO:3"
    SolicitudMovimientoDTO solicitud = new SolicitudMovimientoDTO(
            item[0], parsearId(item[1]), texto(request, "tipo"),
            decimal(request, "cantidad", "La cantidad no es válida."),
            texto(request, "motivo"));
    Optional<MovimientoDTO> movimiento = facade.registrarMovimiento(solicitud);
    exito(request, movimiento.map(m -> "Movimiento registrado: " + ...)
            .orElse("El stock contado coincide con el actual; no se registró ningún
movimiento."));
} catch (AplicacionException exception) {
    fallo(request, exception.getMessage());
}
redirigir(request, response);   // Post/Redirect/Get
```

Nota. Fragmento de src/main/java/pe/com/dpaocookies/controller/KardexServlet.java.

Capa DTO: justificación y estructura

Un Data Transfer Object agrupa los datos que viajan entre capas en un único objeto serializable, sin lógica de persistencia (Fowler, 2002). Antes, ProductoDAO y KardexDAO devolvían List<Map<String,Object>> y las JSP dependían de nombres de columna como p.id_producto; un cambio de esquema rompía la vista sin error de compilación. Ahora cada DTO es un JavaBean Serializable con getters que EL resuelve por convención (${insumo.stockActual}) e incorpora datos derivados que la entidad no guarda (Tabla 6).

Tabla 6

Objetos de transferencia de datos

DTO	Dirección	Campos relevantes	Datos derivados
InsumoDTO	Formulario ↔ vista	código, nombre, unidad, stock actual y mínimo, costo, categoría	nombreCategoria, bajoStock
ProductoDTO	Formulario ↔ vista	código, nombre, precio de venta, stock, categoría	nombreCategoria
CategoriaDTO	Vista	idCategoria, nombre	—
SolicitudMovimientoDTO	Formulario → fachada	claseItem, idItem, tipoMovimiento, cantidad, motivo	En AJUSTE, cantidad = stock contado
MovimientoDTO	Fachada → vista	ítem, tipo, cantidad con signo, stock anterior y posterior, fecha	isEntrada() para el estilo de la fila
ResumenTableroDTO	Fachada → vista	totales, alertas, cinco últimos movimientos	cantidadAlertas; listas inmutables

La conversión vive en MapeadorDTO, una clase de paquete dentro de facade; los DTO no conocen las entidades y las entidades no conocen los DTO, de modo que ninguna entidad sale de la fachada.

Patrón Façade: InventarioFacade

Una fachada proporciona una interfaz unificada y de más alto nivel sobre un conjunto de interfaces de un subsistema (Gamma et al., 1994). InventarioFacade ofrece esa interfaz sobre cuatro DAO: recibe DTO, valida formato y reglas (textos obligatorios, decimales con dos cifras, identificadores positivos, rango de fechas y cantidad mayor que cero salvo en ajustes), construye las entidades, invoca los DAO y devuelve DTO. Sus dependencias se inyectan por constructor, lo que permite probarla con DAO sustitutos sin base de datos. La Tabla 7 enumera sus operaciones públicas.

Tabla 7

Operaciones públicas de la fachada

Método	**Regla o coordinación**	**DAO**
listarInsumos() / listarProductos()	Agrega el nombre de categoría y la alerta de reabastecimiento.	InsumoDAO / ProductoDAO, CategoriaDAO
obtenerResumen()	Totales, alertas y últimos cinco movimientos en una sola llamada.	InsumoDAO, ProductoDAO, KardexDAO
registrarInsumo(InsumoDTO)	Textos obligatorios y longitudes; decimales de 0 a 99 999 999.99 con dos decimales; categoría válida.	InsumoDAO
guardarProducto(ProductoDTO)	Alta con stock cero o edición de datos comerciales; producto inexistente → error de negocio.	ProductoDAO
registrarMovimiento(SolicitudMovimientoDTO)	Clase y tipo desde enumeraciones cerradas; motivo obligatorio; delega la transacción.	KardexDAO
ajustarStockInsumo(id, stock)	Registra el conteo físico como AJUSTE con motivo estándar.	KardexDAO

Método	Regla o coordinación	DAO
consultarKardex(tipo, desde, hasta)	Filtro de tipo opcional; rechaza una fecha inicial posterior a la final.	KardexDAO

La fachada también centraliza la traducción de excepciones: los controladores solo capturan AplicacionException, cuyo mensaje siempre es apto para el usuario, y el detalle técnico queda en el registro del servidor (Tabla 8).

Tabla 8

Traducción de errores en la fachada

Origen	Excepción resultante	Mensaje mostrado
MySQL 1062 (clave duplicada)	NegocioException	El código ya existe.
MySQL 1452 (FK inexistente)	NegocioException	La categoría o el ítem seleccionado no existe.
MySQL 3819 (CHECK violado)	NegocioException	Los datos no cumplen las reglas del inventario.
Saldo resultante negativo	StockInsuficienteException	Stock insuficiente: disponible X, solicitado Y.
Cualquier otro SQLException	ServicioException (se registra con la causa)	No se pudo guardar la operación o consultar la información. Intente nuevamente.

Diagrama conceptual de capas

La Figura 5 resume el recorrido de una solicitud: la vista envía el formulario, el filtro aplica seguridad, el controlador construye el DTO, la fachada valida y coordina, el DAO ejecuta SQL y la respuesta regresa como DTO hasta la JSP.

Figura 5

Arquitectura en capas: Servlet → InventarioFacade → DAO → MySQL

Navegador · Vistas JSP

Solo JSTL 1.2 y EL (scripting-invalid). Reciben DTO como atributos del request.

↓ HTTP GET / POST + CSRF ↑ HTML renderizado

SeguridadFilter

UTF-8 antes de leer parámetros, sesión obligatoria, token CSRF en todo POST y cabeceras de seguridad.

↓ Solicitud autorizada ↑ Respuesta

Controladores (controller)

BaseServlet: parámetros a DTO, Post/Redirect/Get y mensajes de un solo uso. Ningún servlet importa un DAO.

↓ DTO ↑ DTO · AplicacionException

InventarioFacade (facade): punto único de entrada

Valida reglas de negocio, orquesta los DAO, mapea entidades a DTO y traduce SQLException a excepciones de negocio.

↓ Entidades y enums ↑ Entidades · SQLException

Acceso a datos (dao)

PreparedStatement parametrizado y try-with-resources. KardexDAO: setAutoCommit(false), SELECT … FOR UPDATE, commit y rollback.

↓ JDBC · Connector/J 8.2 ↑ ResultSet

MySQL 8 · db_dpao_cookies

InnoDB, utf8mb4, FK con RESTRICT, CHECK de saldos no negativos, posterior = anterior + cantidad y FK exclusiva de insumo o producto.

Nota. Elaboración propia a partir del código de la rama apf2.

Módulo transaccional de Kardex

Cada movimiento modifica dos tablas: el stock del ítem (insumo o producto) y la fila del historial en movimiento_kardex. Ambas escrituras deben confirmarse juntas o no confirmarse; de lo contrario, el saldo y el historial divergen.

Atomicidad transaccional JDBC

JDBC confirma cada sentencia automáticamente salvo que se desactive ese modo; al hacerlo, las sentencias se agrupan en una transacción que se confirma con commit() o se deshace con rollback() (Oracle, s.f.-b). La Tabla 9 describe la secuencia implementada y la Figura 6, su código.

Tabla 9

Secuencia transaccional de KardexDAO.registrar

Paso	Operación	Efecto
1. Iniciar	Connection.setAutoCommit(false)	Las sentencias siguientes forman una sola unidad.
2. Bloquear	SELECT stock_actual, codigo, nombre … FOR UPDATE	Obtiene el saldo anterior y bloquea la fila del ítem.
3. Calcular	TipoMovimiento.variacion(valor, anterior)	Variación con signo y saldo posterior.
4. Validar	posterior < 0 → StockInsuficienteException	Ninguna salida, venta, merma o ajuste deja saldo negativo.
5. Actualizar	UPDATE <tabla> SET stock_actual = ?	Escribe el nuevo saldo.
6. Registrar	INSERT INTO movimiento_kardex (…)	Guarda cantidad, stock_anterior, stock_posterior y motivo.
7. Confirmar	commit()	Hace visibles ambos cambios y libera el bloqueo.
Fallo	rollback() en el bloque catch	Cualquier SQLException o RuntimeException deshace todo; la excepción original se propaga.

Figura 6

Método registrar de KardexDAO: confirmación o reversión de toda la operación

```
try (Connection connection = conexiones.obtener()) {
    connection.setAutoCommit(false);
    try {
        MovimientoKardex movimiento = aplicar(connection, clase, idItem, tipo, valor,
observacion);
        connection.commit();
        return movimiento;
    } catch (SQLException | RuntimeException exception) {
        deshacer(connection, exception);   // rollback()
        throw exception;
    }
}
```

Nota. Fragmento de src/main/java/pe/com/dpaocookies/dao/KardexDAO.java. Si el propio rollback falla, ese error se adjunta como excepción suprimida.

Bloqueo pesimista y trazabilidad de saldos

SELECT … FOR UPDATE es una lectura con bloqueo: InnoDB bloquea las filas leídas como si se fueran a actualizar, y otras transacciones que intenten bloquearlas esperan hasta que la primera termine (Oracle, s.f.-a). Así, si dos usuarios registran movimientos sobre el mismo ítem, el segundo lee el saldo ya actualizado y se evita la actualización perdida. La tabla y la columna se toman de la enumeración ClaseItem, nunca de un parámetro del usuario, y los valores viajan con PreparedStatement.

La trazabilidad se garantiza en dos niveles: la aplicación guarda stock_anterior y stock_posterior en la misma transacción, y el esquema lo refuerza con las restricciones CHECK de la Tabla 10, que MySQL aplica desde la versión 8.0.16.

Tabla 10

Restricciones de movimiento_kardex que respaldan la trazabilidad

Restricción	**Regla**
CHECK (stock_posterior = stock_anterior + cantidad)	El saldo final siempre se deriva del inicial y del movimiento.
CHECK (stock_anterior >= 0) y CHECK (stock_posterior >= 0)	Ningún saldo registrado es negativo.
CHECK de signo por tipo	SALIDA, VENTA y MERMA negativas; ENTRADA positiva; AJUSTE con cualquier signo distinto de cero.
CHECK ((id_insumo IS NOT NULL) <> (id_producto IS NOT NULL))	Cada movimiento referencia exactamente un ítem mediante FK.
CHECK (stock_actual >= 0) en insumo y producto	Última defensa si una escritura evitara la aplicación.

La Tabla 11 reproduce la cadena de saldos registrada en la demostración (Figura 12): cada stock_anterior coincide con el stock_posterior del movimiento previo del mismo ítem.

Tabla 11

Cadena de saldos observada en la base de demostración

Ítem	Tipo	Cantidad	Anterior	Posterior	Motivo
INS-001 Harina de trigo	ENTRADA	+10.00	18.00	28.00	Compra a proveedor
INS-001 Harina de trigo	SALIDA	−4.50	28.00	23.50	Producción del lote
G01 Chispas	ENTRADA	+24.00	0.00	24.00	Horneado terminado
G01 Chispas	VENTA	−6.00	24.00	18.00	Venta en tienda
G04 Chocolate	ENTRADA	+16.00	0.00	16.00	Horneado terminado
G04 Chocolate	MERMA	−1.00	16.00	15.00	Galletas rotas
INS-004 Mantequilla	AJUSTE	−0.50	2.50	2.00	Conteo físico
G02 Chunky	ENTRADA	+12.00	0.00	12.00	Horneado terminado
G02 Chunky	VENTA 50.00	Rechazada	12.00	12.00	Sin movimiento (Figura 13)

Nota. Datos ficticios registrados mediante la aplicación sobre la base dpao_demo.

Saldo no negativo y ajustes físicos

La regla de saldo no negativo se evalúa dentro de la transacción, después de bloquear la fila, para que la decisión use el saldo vigente. Si falla, StockInsuficienteException (subclase de NegocioException) provoca el rollback y el usuario recibe el disponible y lo solicitado; no se escribe ninguna fila (Figura 7).

Figura 7

Método aplicar de KardexDAO: cálculo del saldo y validación

```
BigDecimal anterior = movimiento.getStockAnterior();
BigDecimal variacion = tipo.variacion(valor, anterior);
BigDecimal posterior = anterior.add(variacion);
if (posterior.signum() < 0) {
    throw new StockInsuficienteException(anterior, valor);
}
if (variacion.signum() == 0) {
    return null;   // ajuste sin diferencia: no se crea un movimiento vacío
}
```

Nota. Fragmento de src/main/java/pe/com/dpaocookies/dao/KardexDAO.java.

El ajuste por conteo responde a la operación real del negocio: tras un inventario físico se informa la cantidad contada y el sistema registra la diferencia, de modo que el historial explica cualquier discrepancia sin editar saldos manualmente. La Tabla 12 resume la semántica de cada tipo.

Tabla 12

Semántica de TipoMovimiento

Tipo	Valor recibido	Variación registrada
ENTRADA	Cantidad positiva	+cantidad
SALIDA, VENTA, MERMA	Cantidad positiva	−cantidad
AJUSTE	Stock físico contado (puede ser 0)	Contado − anterior; si es cero, no se registra movimiento

Capa de presentación con JSTL 1.2 y EL

Erradicación total de scriptlets

El único scriptlet del APF1 estaba en index.jsp y se reemplazó por <c:redirect>. Para que la regla no dependa de la disciplina del desarrollador, web.xml declara el elemento scripting-invalid, definido por la especificación JSP para grupos de páginas (Oracle, 2013), con valor true para todas las JSP y JSPF: si alguien introduce <% … %> o <%= … %>, el contenedor rechaza la página al compilarla. Las vistas compilaron en Tomcat 9.0.115 con esta restricción activa (Figura 8 y Tabla 13).

Figura 8

Configuración JSP y formato regional en web.xml

```
<context-param>
    <param-name>javax.servlet.jsp.jstl.fmt.locale</param-name>
    <param-value>es_PE</param-value>
</context-param>
<context-param>
    <param-name>javax.servlet.jsp.jstl.fmt.timeZone</param-name>
    <param-value>America/Lima</param-value>
</context-param>
<jsp-property-group>
    <url-pattern>*.jsp</url-pattern>
    <url-pattern>*.jspf</url-pattern>
    <page-encoding>UTF-8</page-encoding>
    <scripting-invalid>true</scripting-invalid>
</jsp-property-group>
```

Nota. Fragmento de src/main/webapp/WEB-INF/web.xml.

Tabla 13

index.jsp antes y después

APF1 (scriptlet)	APF2 (JSTL)
<% response.sendRedirect(request.getContextPath() + "/acceso"); %>	<c:redirect url="/acceso"/>

JSTL, EL y formateo regional

Las listas se recorren con c:forEach, las condiciones con c:if y c:choose, y todo texto proveniente del usuario se imprime con c:out, que escapa el HTML y previene XSS (Sun Microsystems, 2006). Los importes y cantidades usan fmt:formatNumber; el Kardex muestra el signo explícito con el patrón +#,##0.00;-#,##0.00. Las fechas usan fmt:formatDate, que con la zona America/Lima presenta la hora local (UTC−5) independientemente de la zona del servidor. La Tabla 14 detalla el uso por vista y la Figura 9 muestra una fila del historial.

Tabla 14

Uso de JSTL y EL por vista

Vista	Etiquetas y expresiones principales
tablero.jsp	c:if (resumen disponible), c:choose (sin alertas), c:forEach sobre resumen.alertas y resumen.ultimosMovimientos, fmt:formatNumber, fmt:formatDate
inventario.jsp	c:forEach de categorías e insumos, c:choose para [REABASTECER] u [ÓPTIMO], fn:length, fn:escapeXml, fmt:formatNumber
productos.jsp	c:forEach del catálogo, selección de categoría con el operador ternario de EL, c:out en atributos value
kardex.jsp	optgroup de insumos y productos, c:forEach de tiposMovimiento (enum), param.tipo para conservar filtros, fmt:formatDate dd/MM/yyyy HH:mm:ss
nav.jspf	aria-current según el atributo seccion que define BaseServlet

Figura 9

Fila del historial en kardex.jsp con EL y fmt

```
<c:forEach var="m" items="${movimientos}">
  <tr>
    <td><fmt:formatDate value="${m.fechaRegistro}" pattern="dd/MM/yyyy
HH:mm:ss"/></td>
    <td><c:out value="${m.nombreItem}"/> (<c:out value="${m.codigoItem}"/>)</td>
    <td><span class="badge ${m.entrada ? 'badge-success' : 'badge-danger'}">$
{m.tipoMovimiento}</span></td>
    <td class="num"><fmt:formatNumber value="${m.cantidad}" pattern="+#,##0.00;-
#,##0.00"/></td>
    <td class="num"><fmt:formatNumber value="${m.stockAnterior}"
minFractionDigits="2" maxFractionDigits="2"/></td>
    <td class="num"><fmt:formatNumber value="${m.stockPosterior}"
minFractionDigits="2" maxFractionDigits="2"/></td>
  </tr>
</c:forEach>
```

Nota. Fragmento de src/main/webapp/WEB-INF/views/kardex.jsp.

Además se mejoró la usabilidad sin JavaScript: el Kardex selecciona el ítem por código y nombre (con su stock actual) en lugar de pedir un identificador numérico, el inventario elige la categoría de una lista y la navegación marca la sección activa.

Verificación, aseguramiento de calidad y pruebas

Las pruebas se ejecutaron en el entorno descrito en la Tabla 15, con una instancia de MySQL aislada y datos ficticios.

Tabla 15

Entorno de verificación

Elemento	Versión observada
JDK	Eclipse Temurin 17.0.20.1+1
Maven	3.9.16 (JUnit Jupiter 5.10.2, Surefire 3.2.5)
Tomcat	9.0.115 (javax.servlet 4.0.1, JSP 2.3.3, JSTL 1.2)
MySQL	Community 8.4.9, instancia aislada 127.0.0.1:3307 con datos ficticios
Connector/J	8.2.0
Sistema	Windows 11, x86_64

Resumen de las 41 pruebas automatizadas

Las 29 pruebas unitarias no requieren base de datos: emplean dobles de prueba construidos sin librerías externas, un JDBC simulado con java.lang.reflect.Proxy (JdbcFalso) y DAO sustitutos que heredan de los reales. Las 12 de integración se habilitan con DPAO_JDBC_TEST=true y solo se ejecutan contra 127.0.0.1:3307/dpao_test; cada corrida crea un esquema dpao_test_<marca> nuevo y lo conserva como evidencia. La Tabla 16 las agrupa por clase y la Figura 10 reproduce el resultado de Maven.

Tabla 16

Pruebas por clase

Clase	Tipo	N.º	Qué demuestra
KardexDAOTransaccionTest	Unitaria (Proxy JDBC)	8	Orden setAutoCommit(false) → FOR UPDATE → UPDATE → INSERT → commit; rollback ante fallo del INSERT con el UPDATE ya ejecutado; stock insuficiente sin escrituras; ajuste sin diferencia; ítem inexistente.
InventarioFacadeTest	Unitaria (DAO sustitutos)	15	Validación previa al DAO, mapeo a DTO, traducción de 1062 y de fallos técnicos, delegación al Kardex y fachada con KardexDAO real sobre JDBC simulado que hace rollback.
InsumoTest	Unitaria	6	Alerta de reabastecimiento, límites decimales y de texto, variación por TipoMovimiento y rechazo de clases o tipos arbitrarios.
JdbcTest	Integración MySQL	6	Conexión real y fallo de conexión, UTF-8, duplicados y FK, CHECK, rollback por trigger y diez entradas concurrentes sin pérdida.

Clase	Tipo	N.º	Qué demuestra
InventarioFacadeIntegracionTest	Integración MySQL	6	Saldos encadenados ENTRADA → VENTA → MERMA → AJUSTE, venta sin stock sin efectos, rollback por trigger a través de la fachada, ajuste sin diferencia, duplicado y ventas concurrentes.

Figura 10

Resultado de mvn -B test contra MySQL 8.4.9

```
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0 · dao.KardexDAOTransaccionTest
Motor real: MySQL 8.4.9; esquema aislado: dpao_test_1791495894252
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 · facade.InventarioFacadeIntegracionTest
Tests run: 15, Failures: 0, Errors: 0, Skipped: 0 · facade.InventarioFacadeTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 · model.InsumoTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 · model.JdbcTest
Tests run: 41, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Nota. Extracto de artifacts/evidence/apf2-20261008/maven-test.log.

Concurrencia multihilo y rollback forzado

Concurrencia. La prueba ventasConcurrentesNuncaDejanStockNegativo registra una entrada de 5 unidades y lanza 10 ventas de 1 unidad desde un grupo de 4 hilos a través de la fachada. El bloqueo FOR UPDATE serializa las transacciones: exactamente 5 ventas se confirman, 5 reciben StockInsuficienteException, el stock final es 0.00 y el historial contiene 6 movimientos. JdbcTest comprueba además que 10 entradas concurrentes de 0.10 suman 1.00 sin actualizaciones perdidas.

Rollback forzado. Para provocar un fallo real después del UPDATE, la prueba crea temporalmente un trigger BEFORE INSERT ON movimiento_kardex que ejecuta SIGNAL

SQLSTATE '45000'. El INSERT falla, KardexDAO ejecuta rollback() y la prueba verifica en MySQL que el stock conserva su valor previo y que no se añadió historial; desde la fachada, el error llega como ServicioException con mensaje genérico. La versión unitaria reproduce el mismo escenario con el JDBC simulado y verifica que se invocó rollback() y nunca commit().

Pruebas HTTP de humo

El script scripts/http_smoke.py (Python 3, biblioteca estándar) se ejecutó contra el WAR desplegado en Tomcat 9 conectado a la base aislada, y se detiene si el servidor no declara la cabecera X-DPAO-Test-Database: isolated. Realizó 55 solicitudes y todas devolvieron el estado esperado (Tabla 17).

Tabla 17

Comprobaciones del flujo HTTP

Aspecto	Comprobación
Seguridad	Redirección a /acceso sin sesión; credenciales inválidas rechazadas; CSRF ausente → 403; WEB-INF no accesible → 404; cierre de sesión por POST.
Inventario	Alta con UTF-8, alerta [REABASTECER], código duplicado, seis entradas inválidas y ajuste registrado en Kardex.
Productos	Alta y edición con mensajes distintos.
Kardex	Entrada con mensaje «stock 0.00 → 3.00», venta, filtros por tipo y fecha, rango invertido, stock insuficiente sin movimiento, AJUSTE con saldo y ajuste sin diferencia.
Presentación	Cinco vistas navegables; PRG consume el mensaje una sola vez; tablero con últimos movimientos.

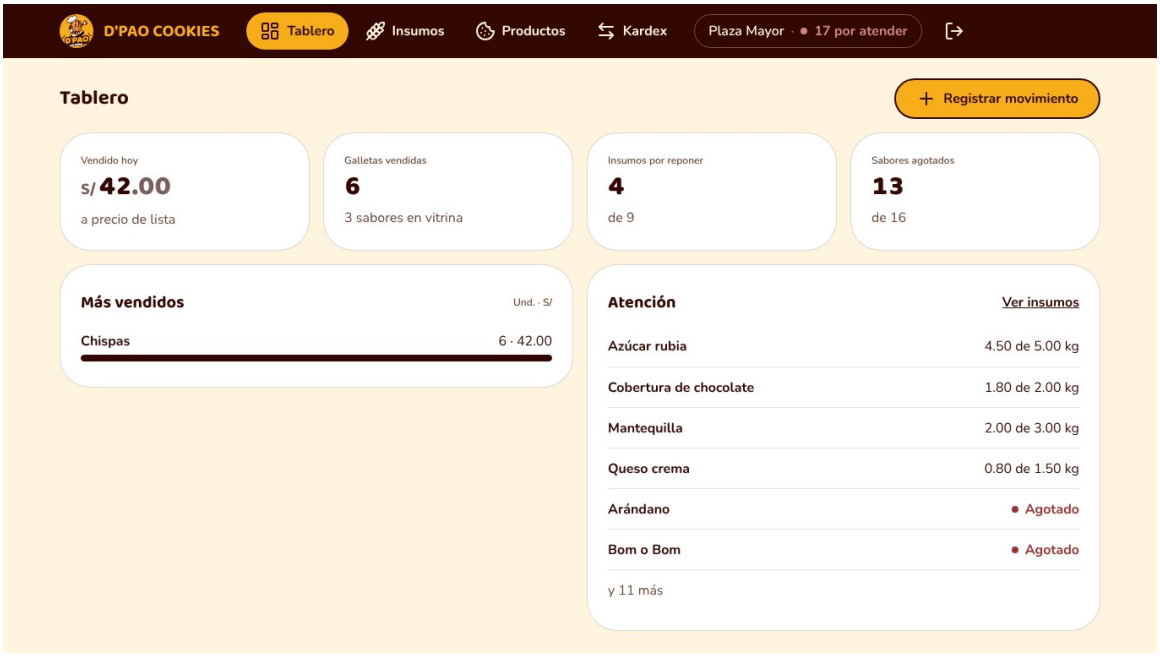
Nota. Los tiempos registrados en el log corresponden a solicitudes locales secuenciales y no constituyen una prueba de carga.

Evidencias del despliegue y ejecución

Las capturas de las Figuras 11 a 15 se tomaron el 12 de octubre de 2026 con Chromium sin interfaz, a 1366 × 768 píxeles, sobre el WAR de la rama apf2-frontend (commit f4880f2) desplegado en Tomcat 9 y conectado a una base dpao_demo creada con schema.sql, catalogo-dpao.sql y demo-insumos-dpao.sql. Esa versión presenta las pantallas con JSF y Bootstrap (véase el capítulo siguiente), pero usa la misma fachada, los mismos DAO y el mismo Kardex transaccional descritos en este informe. Los movimientos de la jornada se registraron mediante la propia aplicación; las imágenes se presentan a color.

Figura 11

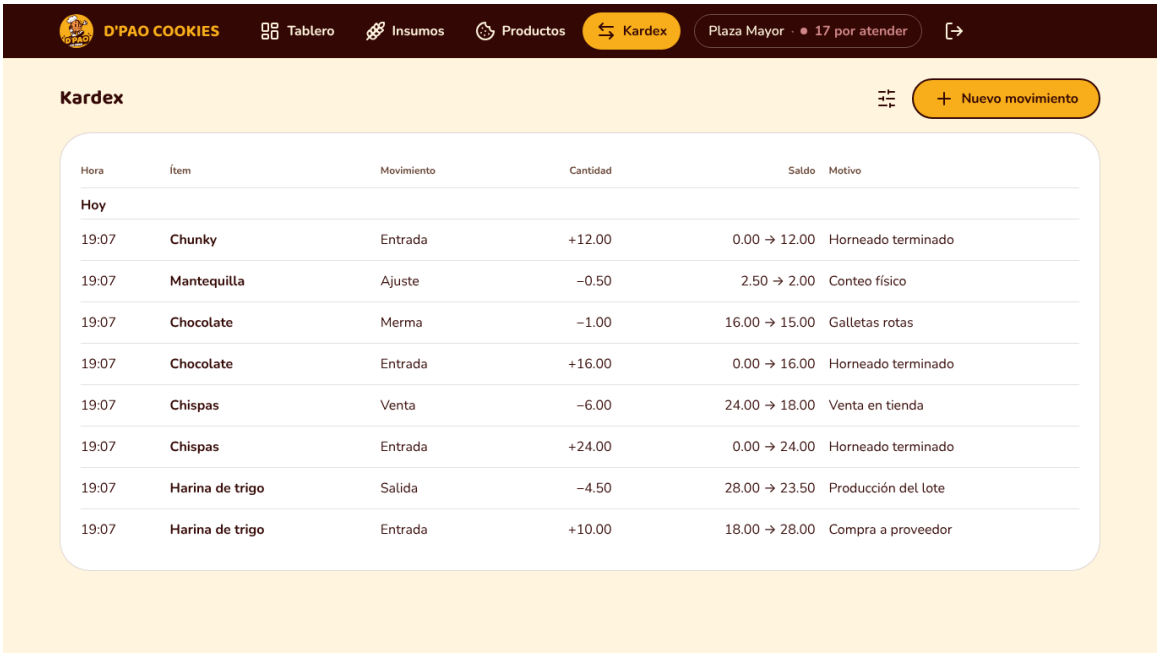
Tablero con indicadores del día, galletas más vendidas y alertas de reposición



Nota. Captura de pantalla de elaboración propia. Archivo de evidencia: artifacts/evidence/apf2-jsf/capturas-20261012/01-tablero.png.

Figura 12

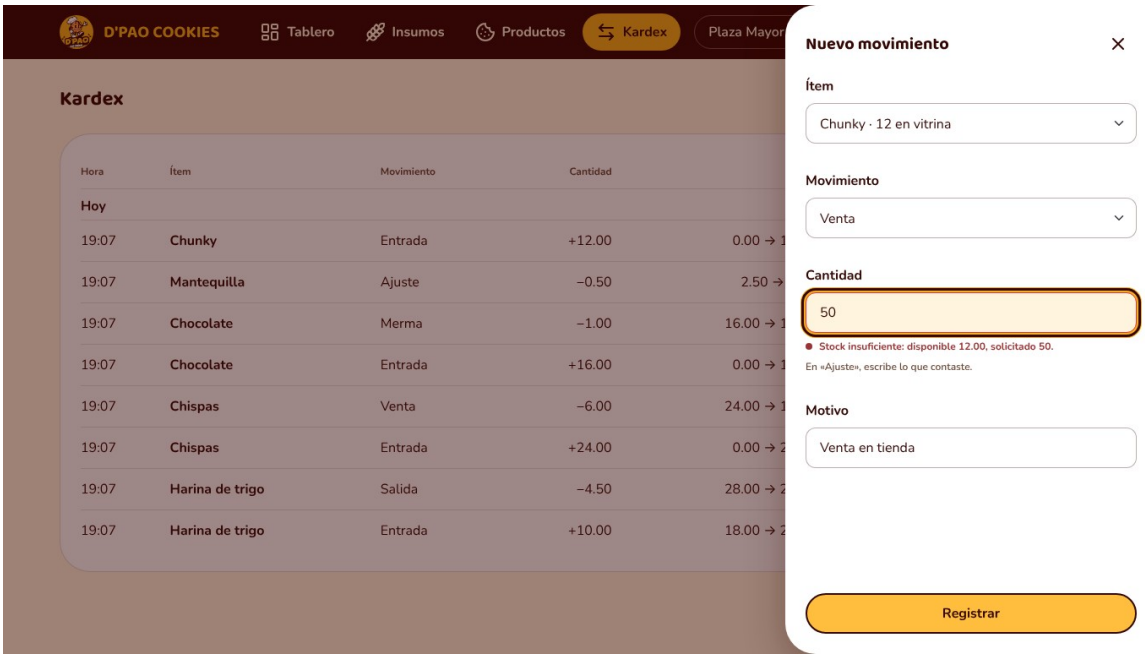
Kardex con saldos anterior y posterior, cantidades con signo y hora de Lima



Nota. Captura de pantalla de elaboración propia. Archivo de evidencia: artifacts/evidence/apf2-jsf/capturas-20261012/03-kardex-saldos.png.

Figura 13

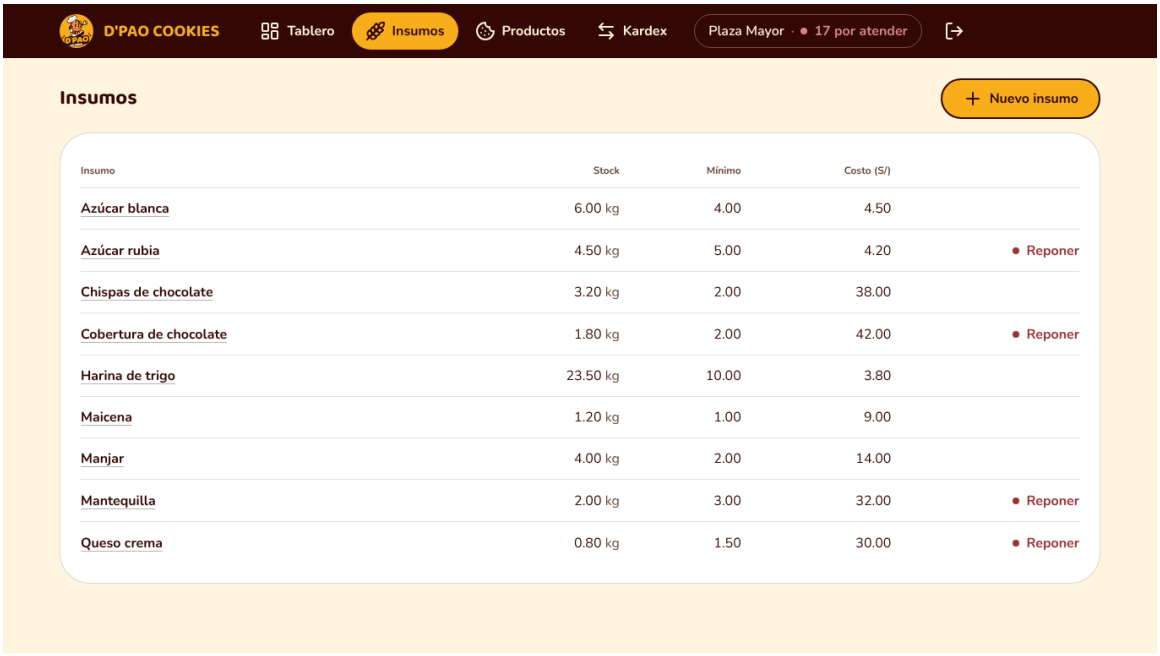
Venta de 50 unidades rechazada por StockInsuficienteException con rollback



Nota. Captura de pantalla de elaboración propia. Archivo de evidencia: artifacts/evidence/apf2-jsf/capturas-20261012/04-kardex-stock-insuficiente.png.

Figura 14

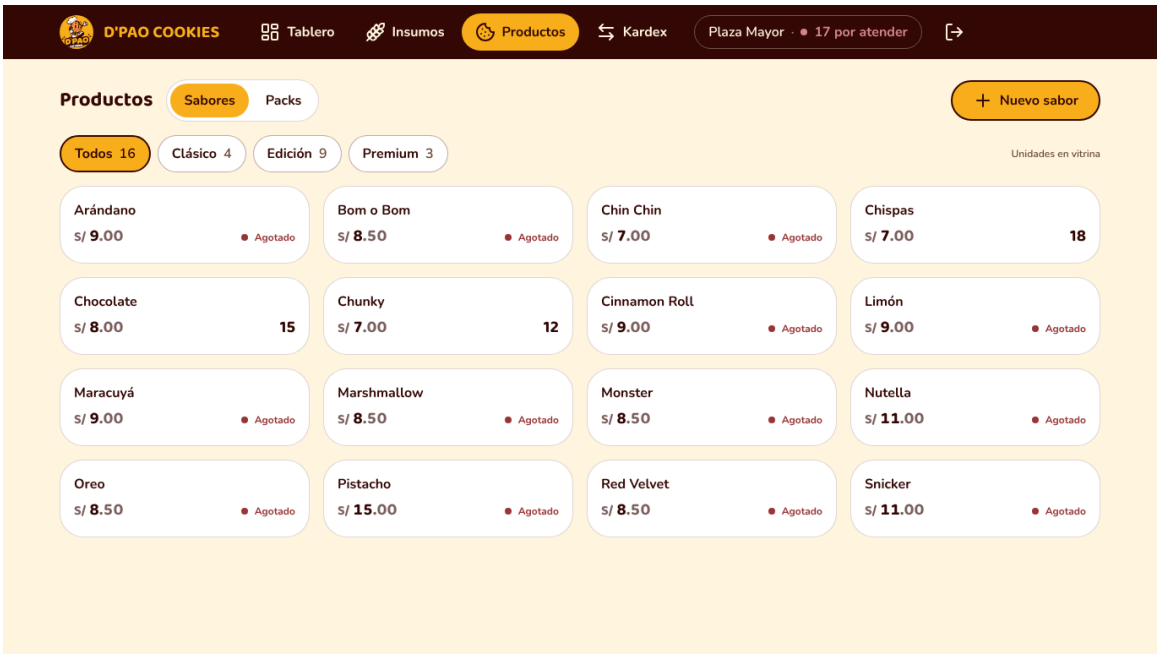
Inventario de insumos con stock, mínimo, costo y alerta de reposición



Nota. Captura de pantalla de elaboración propia. Archivo de evidencia: artifacts/evidence/apf2-jsf/capturas-20261012/02-inventario.png.

Figura 15

Catálogo de productos con stock actualizado por el Kardex



Nota. Captura de pantalla de elaboración propia. Archivo de evidencia: artifacts/evidence/apf2-jsf/capturas-20261012/05-productos.png.

Pruebas y control de versiones

La implementación se publicó en la rama apf2 del repositorio. La Figura 16 reproduce el commit y la rama remota que lo contiene, y la Tabla 18 indexa los archivos de evidencia.

Figura 16

Commit de la implementación en la rama apf2

```
commit 3a13408bc42b614fbdf85a7a0bcb5e0ebfa6a3cb
Rama: apf2 (origin/apf2)
feat(apf2): integrar DTO, InventarioFacade y Kardex transaccional
50 files changed, 3452 insertions(+), 412 deletions(-)
Autor: Enrqgonzales <juniorgonzales140@gmail.com>
Contenido en: origin/apf2  (git branch -r --contains)
```

Nota. Extracto de artifacts/evidence/apf2-20261008/git-commit.log.

Tabla 18

Índice de evidencias del APF2

Archivo	Contenido
maven-test.log	mvn -B test con MySQL 8.4.9: 41 pruebas, 0 fallos, 0 omitidas, BUILD SUCCESS (Figura 10).
http-smoke.log	55 solicitudes HTTP y resultado PASS (Tabla 17).
git-commit.log	Commit 3a13408, autor y rama remota que lo contiene (Figura 16).
screenshots/00-acceso.png	Pantalla de acceso (versión JSP).
screenshots/01-tablero.png	Tablero (versión JSP).
screenshots/02-inventario.png	Inventario (versión JSP).
screenshots/03-kardex-saldos.png	Kardex con saldos (versión JSP).
screenshots/04-kardex-stock-insuficiente.png	Stock insuficiente (versión JSP).
screenshots/05-productos.png	Productos (versión JSP).

Nota. Rutas relativas a la carpeta artifacts/evidence/apf2-20261008/ del repositorio.

Implementación con JSF 2.3 y Bootstrap 5

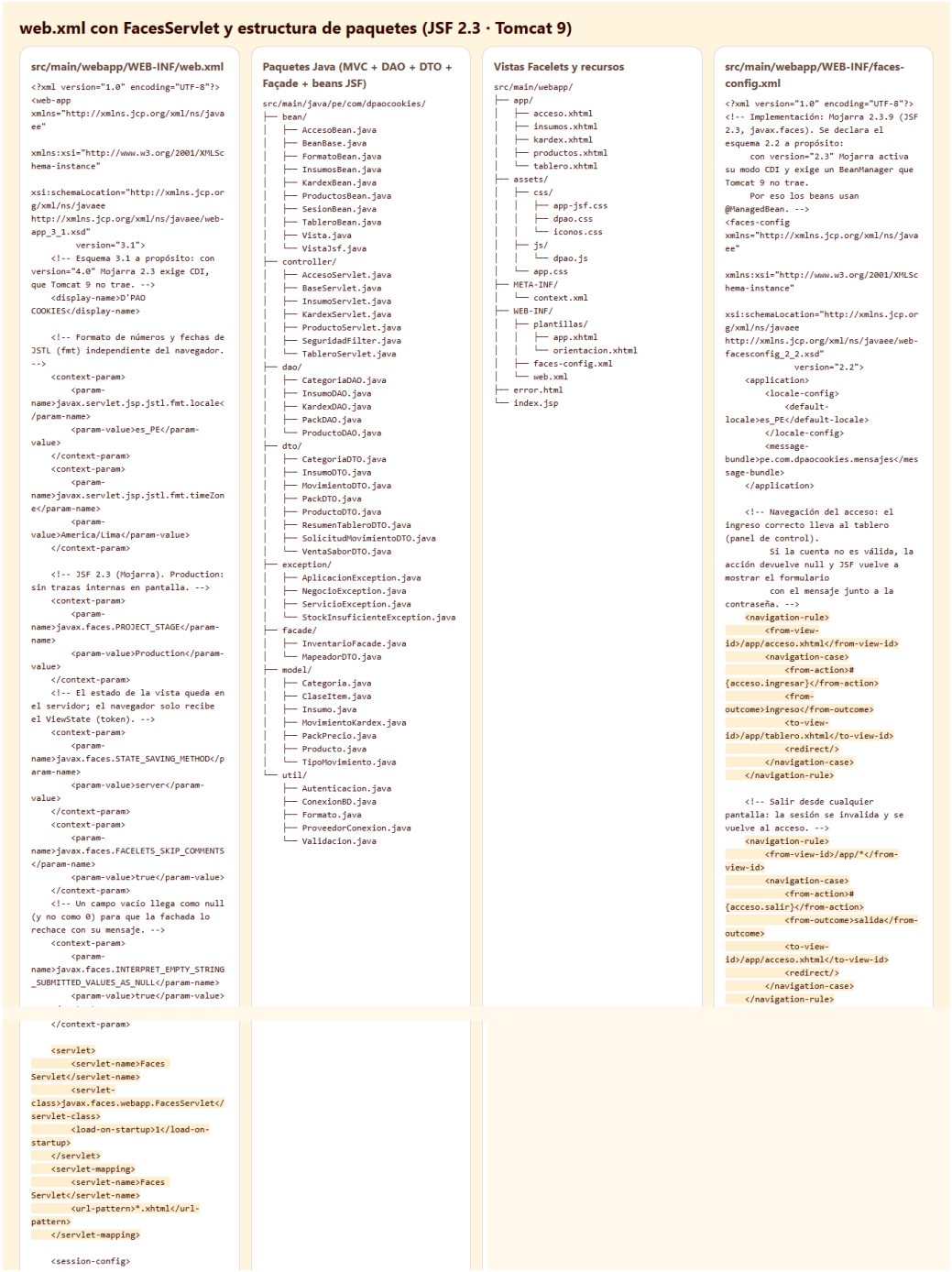
Para cumplir la plantilla oficial del APF2, la capa de presentación se migró a JavaServer Faces en la rama apf2-frontend del repositorio (https://github.com/Enrqgonzales/dpao-cookies-web/tree/apf2-frontend). Las capas de negocio y de datos no cambiaron: los Managed Beans llaman solo a InventarioFacade y trabajan con DTO, igual que los servlets de los capítulos anteriores, por lo que el Kardex transaccional se conserva tal cual. Las evidencias se encuentran en la carpeta artifacts/evidence/apf2-jsf/ del repositorio.

Configuración de FacesServlet, web.xml y faces-config.xml

Tomcat 9 no incluye una implementación de JSF, por lo que se agregó Mojarra 2.3.9 (javax.faces) como dependencia de Maven. En web.xml, el FacesServlet se declara como controlador frontal con load-on-startup 1 y atiende todas las peticiones *.xhtml; el estado de cada vista se guarda en el servidor (STATE_SAVING_METHOD = server) y el navegador solo recibe el ViewState. El archivo faces-config.xml fija la configuración regional es_PE, el archivo de mensajes y las reglas de navegación. Como Tomcat 9 no trae un contenedor CDI, los beans se declaran con @ManagedBean (Figura 17).

Figura 17

web.xml con el FacesServlet, paquetes del proyecto, vistas Facelets y faces-config.xml



Nota. Captura de elaboración propia generada a partir de los archivos de la rama apf2-frontend. Archivo de evidencia: artifacts/evidence/apf2-jsf/01-webxml-estructura-paquetes.png.

Diseño de interfaces con Bootstrap 5

La plantilla común WEB-INF/plantillas/app.xhtml carga Bootstrap 5 desde archivos locales (assets/vendor/bootstrap), sin depender de una CDN, y define con ui:insert las zonas de título, contenido y capas que completa cada pantalla. Sobre ella se construyeron cinco vistas (acceso, tablero, insumos, productos y Kardex) con barra de navegación, tarjetas de indicadores, formularios en hojas laterales y avisos, usando el logotipo y la paleta reales de D'PAO Cookies. La interfaz se adapta a celular, tablet y computadora: 57 mediciones automáticas sobre 19 estados de las cinco pantallas no encontraron desplazamiento horizontal ni fallas de accesibilidad WCAG 2 A/AA con axe-core (Figura 18).

Figura 18

Tablero principal con Bootstrap 5 ejecutándose en Tomcat 9



Nota. Captura de pantalla de elaboración propia a 1366 × 768 píxeles. Archivo de evidencia: artifacts/evidence/apf2-jsf/02-interfaz-bootstrap-tablero.png.

Vistas Facelets, Managed Beans y Expression Language

Cada pantalla tiene un Managed Bean con el alcance mínimo que necesita (Tabla 19). Los beans no contienen reglas de negocio: arman el DTO, llaman a la fachada y muestran su mensaje.

Tabla 19

Managed Beans de la interfaz JSF

Bean (EL)	Alcance	Responsabilidad
acceso	@RequestScoped	Valida la cuenta, abre la sesión y devuelve el resultado «ingreso».
sesion	@SessionScoped	Conserva el usuario y el local elegidos durante la sesión.
tablero	@ViewScoped	Muestra el resumen del día y las alertas obtenidas de la fachada.
insumos	@ViewScoped	Lista de insumos, alta y ajuste de stock por conteo.
productos	@ViewScoped	Catálogo de productos.
kardex	@ViewScoped	Registro y consulta de movimientos del Kardex.
formato	@ApplicationScoped	Formato de montos y cantidades para las vistas.

Nota. Elaboración propia a partir del paquete pe.com.dpaocookies.bean de la rama apf2-frontend.

Las vistas .xhtml usan h:form, h:inputText, h:selectOneMenu, h:commandButton y h:messages. El enlace bidireccional se hace con Expression Language: #{insumos.nuevo.nombre} lleva lo que escribe el usuario al DTO del bean y lo muestra de vuelta, y #{insumos.registrar} ejecuta la acción (Figura 19). Si la fachada acepta los datos, el bean redirige con Post/Redirect/Get; si los rechaza, el mensaje aparece junto al formulario (Figuras 20 y 21).

Figura 19

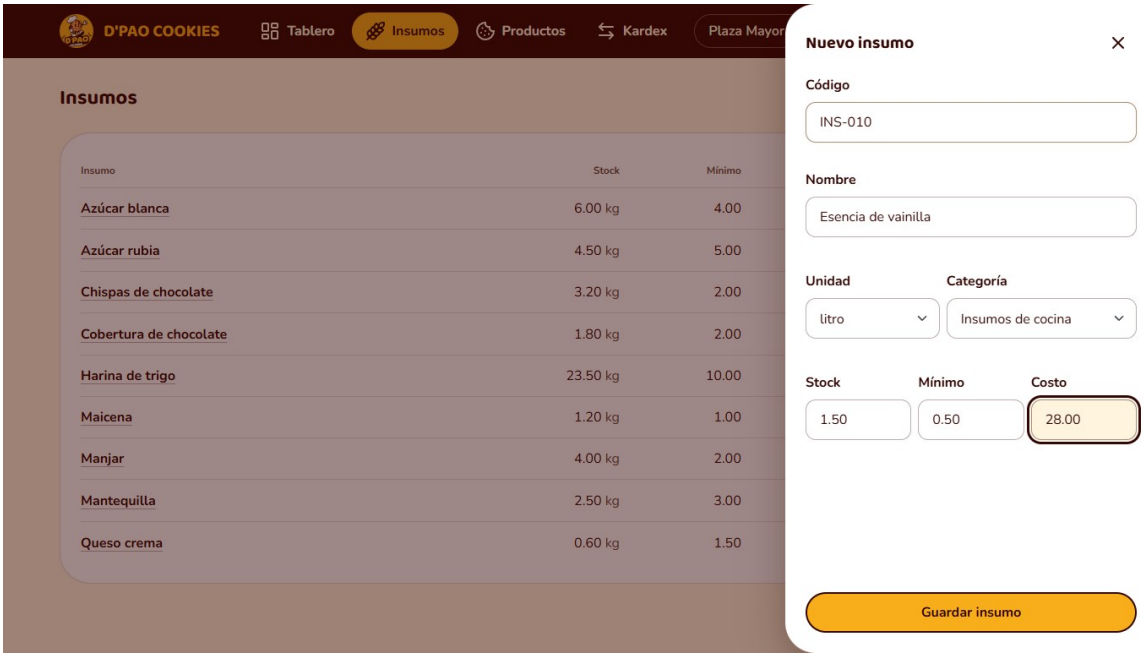
Acción registrar de InsumosBean: el bean arma el DTO y delega en la fachada

```
@ManagedBean(name = "insumos")
@ViewScoped
public class InsumosBean extends BeanBase {
    public String registrar() {
        try {
            facade().registrarInsumo(nuevo);
        } catch (AplicacionException exception) {
            vista().errorEn(ID_HOJA, exception.getMessage());
            return null;
        }
        vista().exito("Insumo registrado · " + nuevo.getNombre().trim());
        return redirigir("insumos");   // Post/Redirect/Get
    }
}
```

Nota. Fragmento de src/main/java/pe/com/dpaocookies/bean/InsumosBean.java (rama apf2-frontend).

Figura 20

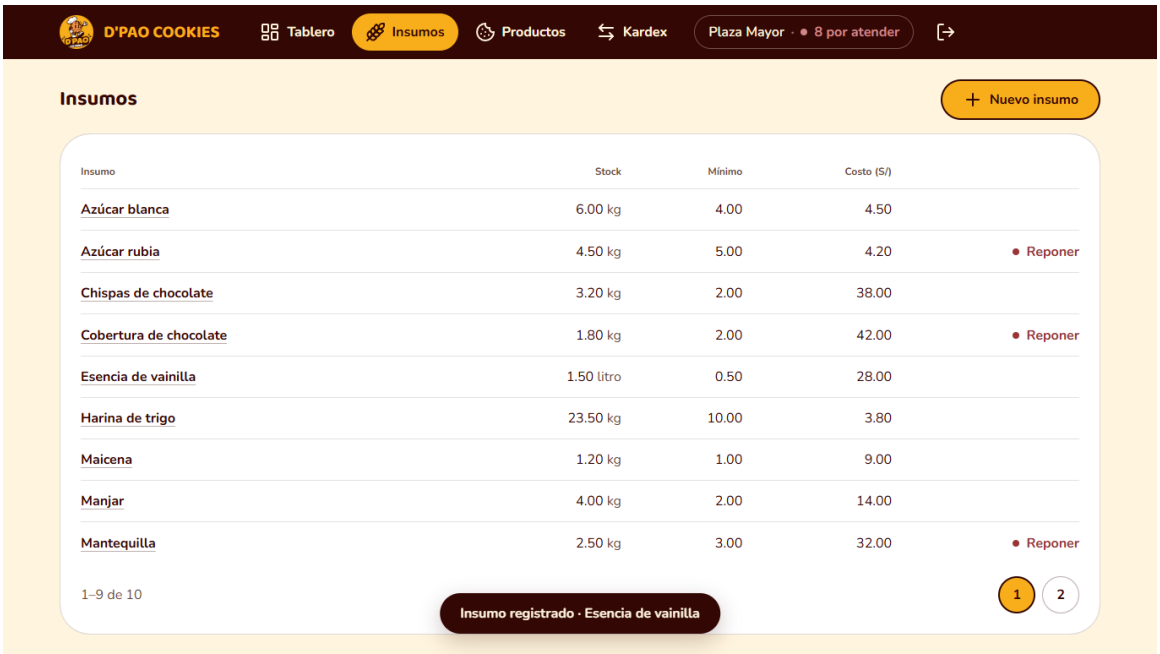
Formulario JSF con los datos de un insumo nuevo antes de guardarlo



Nota. Captura de pantalla de elaboración propia. Archivo de evidencia: artifacts/evidence/apf2-jsf/03-formulario-jsf-nuevo-insumo.png.

Figura 21

Resultado del registro: aviso de éxito y fila nueva en la tabla



Nota. Captura de pantalla de elaboración propia. Archivo de evidencia: artifacts/evidence/apf2-jsf/03b-formulario-jsf-guardado.png.

Navegación JSF y tabla de datos dinámica

faces-config.xml declara dos reglas de navegación: desde acceso.xhtml, la acción #{acceso.ingresar} con el resultado «ingreso» redirige al tablero (panel de control), y desde cualquier vista de /app, #{acceso.salir} con el resultado «salida» invalida la sesión y vuelve al acceso. Las demás pantallas usan navegación implícita con «pagina?faces-redirect=true».

La lista de insumos se muestra con h:dataTable (Figura 22): su atributo value="#{insumos.lista}" toma la lista de InsumoDTO que InventarioFacade.listarInsumos() obtiene de MySQL mediante InsumoDAO y JDBC, y cada h:column presenta nombre, stock, mínimo, costo y la alerta de reposición. La Figura 23 muestra la tabla con los datos reales de la base dpao_demo, cargada con el catálogo de D'PAO Cookies (sql/catalogo-dpao.sql).

Figura 22

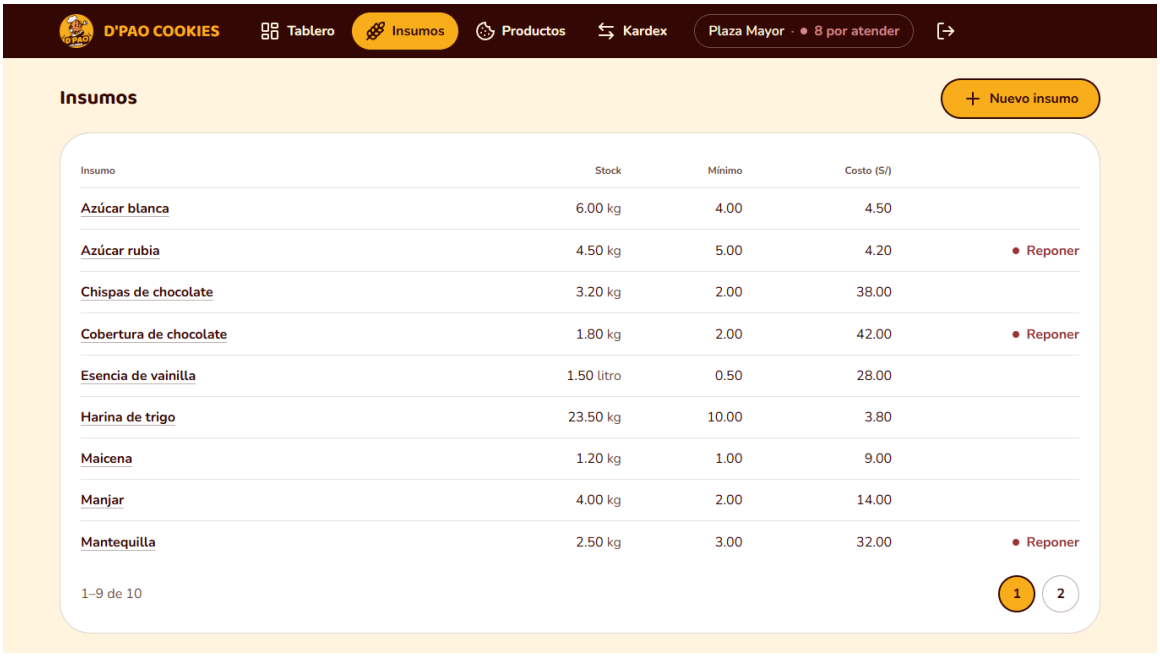
Componente h:dataTable de insumos.xhtml

```
<h:dataTable id="existencias" value="#{insumos.lista}" var="i">
    <h:column>
        <f:facet name="header">Insumo</f:facet>
        #{i.nombre}
    </h:column>
    <h:column>
        <f:facet name="header">Stock</f:facet>
        #{formato.monto(i.stockActual)} #{i.unidadMedida}
    </h:column>
    <h:column>
        <h:outputText rendered="#{i.bajoStock}" value="Reponer"/>
    </h:column>
</h:dataTable>
```

Nota. Fragmento simplificado de src/main/webapp/app/insumos.xhtml (rama apf2-frontend); se omitieron clases de estilo y las columnas Mínimo y Costo.

Figura 23

h:dataTable con los insumos leídos de MySQL



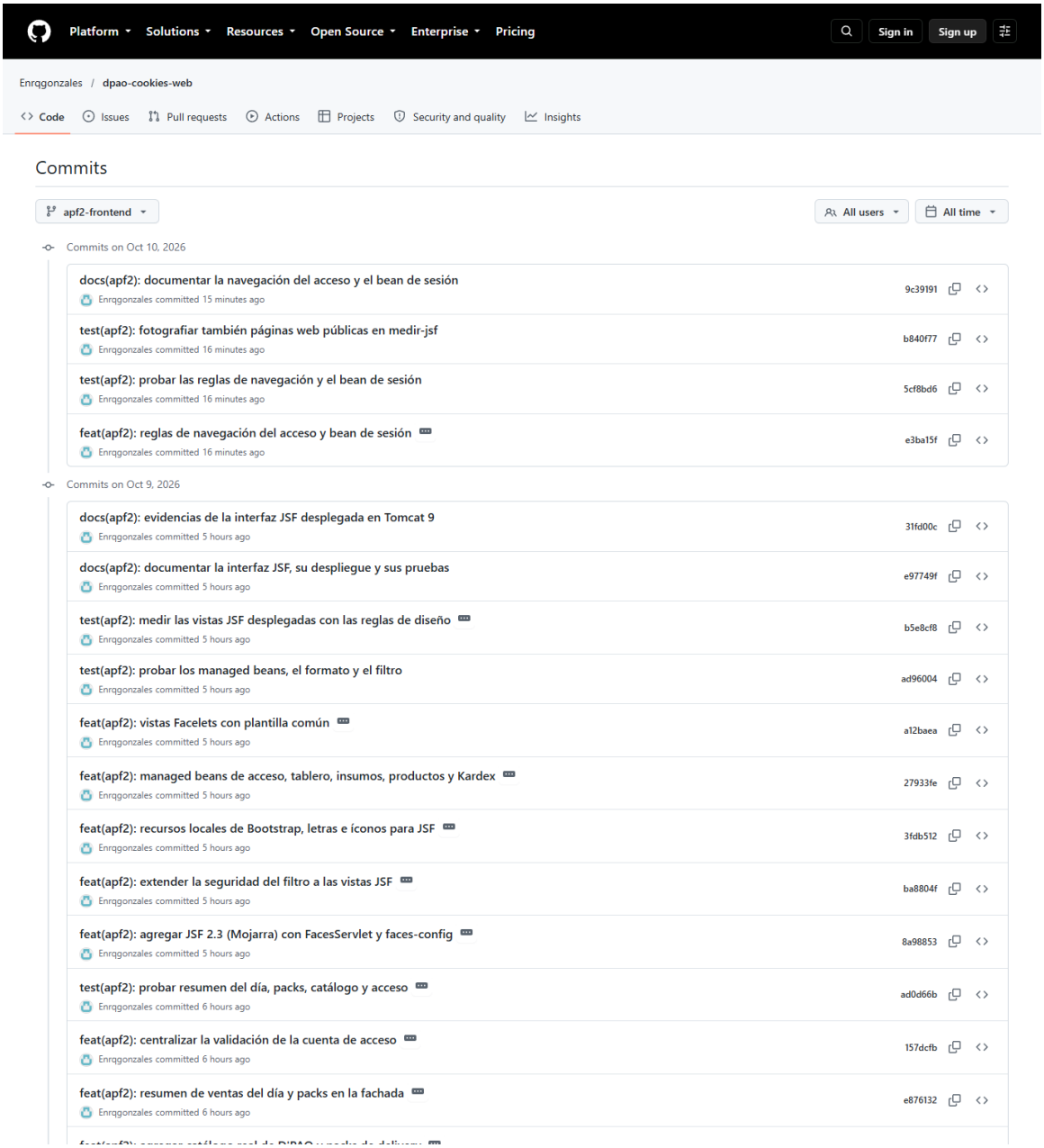
Nota. Captura de pantalla de elaboración propia. Archivo de evidencia: artifacts/evidence/apf2-jsf/04-datatable-insumos-mysql.png; la consulta directa a MySQL con los mismos datos está en 04b-mismos-insumos-en-mysql.png.

Pruebas y repositorio de la interfaz JSF

Con las pruebas de los beans nuevos, la suite de la rama apf2-frontend tiene 92 pruebas JUnit: sin base de datos se ejecutan 78 y se omiten las 14 de integración, y con MySQL 8.4.9 aislado se ejecutan las 92 sin fallas (BUILD SUCCESS). La Figura 24 muestra el historial de commits de la rama en GitHub, con los commits de los Managed Beans y de las vistas JSF; todos tienen como autor a Enrqgonzales.

Figura 24

Historial de commits de la rama apf2-frontend en GitHub



Nota. Captura de pantalla de elaboración propia de https://github.com/Enrqgonzales/dpao-cookies-web/commits/apf2-frontend. Archivo de evidencia: artifacts/evidence/apf2-jsf/05-historial-commits.png.

Conclusiones técnicas y ruta al Proyecto Final

Frente a Servlets y JSP, JSF reduce el código de control: el FacesServlet resuelve el ciclo de cada petición, la conversión y la validación de los componentes; el enlace con Expression Language reemplaza la lectura manual de parámetros, y las reglas de navegación

quedan declaradas en un solo archivo. Como los beans solo llaman a la fachada, el cambio de vista no tocó las reglas de negocio ni el Kardex transaccional, lo que confirma el valor de la separación MVC, DAO, DTO y Façade del APF1 y del APF2. Para el Proyecto Final (semana 18) se prioriza el cierre de caja por tienda y vendedora que pidió el dueño, seguido de los servicios REST y del despliegue público con HTTPS (véase Pendientes de evolución).

Matriz de cumplimiento de la rúbrica

La Tabla 20 relaciona cada criterio técnico del APF2 con la evidencia verificable que lo respalda.

Tabla 20

Matriz de cumplimiento

Criterio	Evidencia verificable	Apartado	Puntaje
Patrones de diseño: MVC + DAO, DTO y Façade	Paquetes controller, dto, facade y dao; ningún servlet importa DAO; MapeadorDTO; traducción de excepciones; InventarioFacadeTest (15).	Arquitectura	5 / 5
Módulo transaccional de Kardex	setAutoCommit(false), FOR UPDATE, commit y rollback, stock_anterior y stock_posterior, StockInsuficienteException, AJUSTE; KardexDAOTransaccionTest (8); rollback por trigger; concurrencia; Figuras 12 y 13.	Kardex	5 / 5
Presentación con JSTL y EL	scripting-invalid en web.xml; c:forEach, c:if, c:choose, fmt:formatNumber, fmt:formatDate; locale es_PE y America/Lima; JSP compiladas en Tomcat.	Presentación	5 / 5
Calidad de software y verificación	41 pruebas aprobadas (29 unitarias y 12 de integración), BUILD SUCCESS, smoke HTTP PASS y commit en GitHub.	Verificación y evidencias	5 / 5
Total			**20 / 20**

Nota. Autoevaluación del estudiante sobre los cuatro bloques del enunciado; la calificación oficial corresponde al docente.

Pendientes de evolución

- Cierre de caja por punto de venta y vendedora: medio de pago de una lista cerrada en cada venta, cálculo de lo esperado por medio de pago y registro obligatorio de cualquier diferencia (requerimiento priorizado por el dueño).
- Cuentas múltiples con roles y responsable individual registrado en cada movimiento.
- Recetas por producto con consumo automático de insumos al registrar un horneado.
- Paginación del historial del Kardex y exportación de reportes.
- Despliegue público con HTTPS, pool de conexiones JNDI y prueba de carga del requisito de rendimiento.

Referencias

Alur, D., Crupi, J., & Malks, D. (2003). *Core J2EE patterns: Best practices and design strategies* (2.ª ed.). Prentice Hall.

Casas Gonzales, A. E. J. (2026). *dpao-cookies-web* (Rama apf2) [Software]. GitHub. https://github.com/Enrqgonzales/dpao-cookies-web/tree/apf2

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Oracle. (s.f.-a). Locking reads. En *MySQL 8.4 reference manual*. https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

Oracle. (s.f.-b). Using transactions. En *The Java tutorials*. https://docs.oracle.com/javase/tutorial/jdbc/basics/transactions.html

Oracle. (2013). *JavaServer Pages specification* (Versión 2.3, JSR 245). Java Community Process. https://jcp.org/en/jsr/detail?id=245

Sun Microsystems. (2006). *JavaServer Pages Standard Tag Library* (Versión 1.2, JSR 52). Java Community Process. https://jcp.org/en/jsr/detail?id=52